ELECTRONIC SALES

NumPy Case Study

Statistical Analysis using NumPy | Python

Dataset: Electronic_Sales.csv | Library: NumPy Only

Contents

MODULE · LOAD DATASET

PART 1

```
import numpy as np
import csv

# Load DataSet
sales_amount    = []
qty_list        = []
price_list      = []
month_list      = []
hour_list       = []
product_list    = []
city_list       = []

with open("/content/Electronic_Sales.csv", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        try:
            sales_amount.append(float(row["Sales"]))
            qty_list.append(int(row["Quantity Ordered"]))
            price_list.append(float(row["Price Each"]))
            month_list.append(int(row["Month"]))
            hour_list.append(int(row["Hour"]))
            product_list.append(row["Product"].strip())
            city_list.append(row["City"].strip())
        except:
            pass
```

```
sales   = np.array(sales_amount)
qty     = np.array(qty_list)
price   = np.array(price_list)
month   = np.array(month_list)
hour    = np.array(hour_list)
product = np.array(product_list)
city    = np.array(city_list)
N       = len(sales)

def header(title):
    print("\n" + "="*68)
    print(f"  {title}")
    print("="*68)

def subheader(title):
    print("\n" + "-"*68)
    print(f"  {title}")
    print("-"*68)

print(f"Dataset loaded! Total records: {N:,}\n")
```

KEY FINDING

→ Successfully loaded and cleaned electronic sales data using NumPy and CSV..
→ Converted raw sales data into structured NumPy arrays for faster analysis.
→ Extracted important business factors like Sales, Quantity, Product, City, Month, and Hour for analysis.
→ Removed invalid/missing records using try-except to improve data accuracy and reliability.

MODULE 1 · DATASET OVERVIEW

PART 1

CODE

```
# ──────────────────────────────────────────
# MODULE 1 — DATASET OVERVIEW
# ──────────────────────────────────────────
header(" MODULE 1 · DATASET OVERVIEW")

print(f"""
  Total Transactions : {N:,}
  Unique Products    : {len(np.unique(product))}
  Unique Cities      : {len(np.unique(city))}
  Months Covered     : January to December (2019)
  Price Range        : ${np.min(price):.2f}  –  ${np.max(price):.2f}
  Sales Range        : ${np.min(sales):.2f}  –  ${np.max(sales):.2f}
  Quantity Range     : {np.min(qty)}  –  {np.max(qty)} units
""")

print("  All Products in Dataset:")
for i, p in enumerate(np.unique(product), 1):
    print(f"    {i:>2}. {p}")
```

OUTPUT

```
═══════════════════════════════════════════════════════════════
  MODULE 1 · DATASET OVERVIEW
═══════════════════════════════════════════════════════════════

Total Transactions : 185,950
Unique Products    : 19
Unique Cities      : 9
Months Covered     : January to December (2019)
Price Range        : $2.99  –  $1700.00
Sales Range        : $2.99  –  $3400.00
Quantity Range     : 1  –  9 units

All Products in Dataset:
   1. 20in Monitor
   2. 27in 4K Gaming Monitor
   3. 27in FHD Monitor
   4. 34in Ultrawide Monitor
   5. AA Batteries (4-pack)
   6. AAA Batteries (4-pack)
   7. Apple Airpods Headphones
   8. Bose SoundSport Headphones
   9. Flatscreen TV
  10. Google Phone
  11. LG Dryer
  12. LG Washing Machine
  13. Lightning Charging Cable
  14. Macbook Pro Laptop
  15. ThinkPad Laptop
  16. USB-C Charging Cable
  17. Vareebadd Phone
  18. Wired Headphones
  19. iPhone
```

KEY FINDING

→ The dataset contains total transaction records from electronic sales data collected throughout 2019
→ Multiple unique products and cities were analyzed to understand product performance and regional sales trends.
→ The dataset includes important business metrics such as Sales, Price, and Quantity ranges.
→ NumPy functions like np.unique(), np.min(), and np.max() were used for statistical overview and data exploration.

MODULE 2 · MEASURE OF CENTERAL TENDENCY

PART 1

CODE

```
subheader("MODULE 1 · Measures of Central Tendency")

print("""
   WHAT IS CENTRAL TENDENCY?
     It tells us WHERE the center of our data is Three ways to find the center:
       → MEAN   = average of all values
       → MEDIAN = middle value when sorted
       → MODE   = most frequently appearing value
""")

# --- MEAN ---
mean_sales = np.mean(sales)
mean_price = np.mean(price)
mean_qty   = np.mean(qty)

print("  ┌── MEAN (Average) ──────────────────────────────────────────")
print(f"  │  Formula : Sum of all values ÷ Total count")
print(f"  │  Mean Sales     = ${mean_sales:,.2f}")
print(f"  │  Mean Price     = ${mean_price:,.2f}")
print(f"  │  Mean Quantity = {mean_qty:.2f} units")
print(f"  │")
print(f"  │   ! Interpretation:")
print(f"  │     On average, each order was worth ${mean_sales:.2f}")
print(f"  └────────────────────────────────────────────────────────────\n")
```

```
# --- MEDIAN ---
median_sales = np.median(sales)
median_price = np.median(price)

print("  ┌── MEDIAN (Middle Value) ──────────────────────────────────")
print(f"  │  Formula : Sort data → pick middle value")
print(f"  │  Median Sales  = ${median_sales:,.2f}")
print(f"  │  Median Price  = ${median_price:,.2f}")
print(f"  │")
print(f"  │  ! Interpretation:")
print(f"  │     Half of orders are below ${median_sales:.2f}")
print(f"  │     Mean (${mean_sales:.2f}) >> Median (${median_sales:.2f})")
print(f"  │     → Data is RIGHT-SKEWED (few very large orders)")
print(f"  └────────────────────────────────────────────────────────────\n")


# --- MODE ---
# For Sales: round to nearest integer and find most common
sales_rounded = np.round(sales, 0).astype(int)
unique_vals, counts = np.unique(sales_rounded, return_counts=True)
mode_sales = unique_vals[np.argmax(counts)]
mode_count = np.max(counts)

products_uniq, prod_counts = np.unique(product, return_counts=True)
mode_product = products_uniq[np.argmax(prod_counts)]

print("  ┌── MODE (Most Frequent Value) ──────────────────────────")
print(f"  │  Formula : Count each value → find most repeated")
print(f"  │  Mode of Sales  = ${mode_sales}  (appears {mode_count:,} times)")
print(f"  │  Mode of Product= {mode_product}")
print(f"  │                   ({np.max(prod_counts):,} orders)")
print(f"  │")
print(f"  │  ! Interpretation:")
print(f"  │     ${mode_sales} is the most common order value")
print(f"  │     (USB-C Charging Cable = most ordered product)")
print(f"  └────────────────────────────────────────────────────────────")
```

MODULE 2 · MEASURE OF CENTERAL TENDENCY

PART 1

OUTPUT

```
============================================================
  PART 1 — DESCRIPTIVE STATISTICS
============================================================

------------------------------------------------------------
  MODULE 1 · Measures of Central Tendency
------------------------------------------------------------

  WHAT IS CENTRAL TENDENCY?
    It tells us WHERE the center of our data is Three ways to find the center:
      → MEAN   = average of all values
      → MEDIAN = middle value when sorted
      → MODE   = most frequently appearing value

  ┌── MEAN (Average) ──────────────────────────────────
  │ Formula : Sum of all values ÷ Total count
  │ Mean Sales    = $185.49
  │ Mean Price    = $184.40
  │ Mean Quantity = 1.12 units
  │
  │  ! Interpretation:
  │    On average, each order was worth $185.49
  └────────────────────────────────────────────────────
```

```
  ┌── MEDIAN (Middle Value) ───────────────────────────
  │ Formula : Sort data → pick middle value
  │ Median Sales  = $14.95
  │ Median Price  = $14.95
  │
  │ ! Interpretation:
  │    Half of orders are below $14.95
  │    Mean ($185.49) >> Median ($14.95)
  │    → Data is RIGHT-SKEWED (few very large orders)
  └────────────────────────────────────────────────────

  ┌── MODE (Most Frequent Value) ──────────────────────
  │ Formula : Count each value → find most repeated
  │ Mode of Sales  = $12  (appears 38,937 times)
  │ Mode of Product= USB-C Charging Cable
  │                  (21,903 orders)
  │
  │ ! Interpretation:
  │    $12 is the most common order value
  │    (USB-C Charging Cable = most ordered product)
  └────────────────────────────────────────────────────
```

KEY FINDING

→ Calculated Mean, Median, and Mode to understand the central behavior of sales data.
→ Average order value and product price were identified using np.mean().
→ Comparison of Mean and Median showed the sales data is right-skewed due to some very large orders..

MODULE 3 · MEASURES OF DISPERSION

PART 1

CODE

```
subheader("MODULE 2 · Measures of Dispersion")

print("""
   📖 WHAT IS DISPERSION?
      It tells us HOW SPREAD OUT the data is.
      Even if two datasets have the same mean,
      their spread can be very different!
""")

# --- RANGE ---
range_sales = np.max(sales) - np.min(sales)
print("  ┌── RANGE ──────────────────────────────────────────────────")
print(f"  │  Formula : Max − Min")
print(f"  │  Max Sales   = ${np.max(sales):,.2f}")
print(f"  │  Min Sales   = ${np.min(sales):,.2f}")
print(f"  │  Range       = ${range_sales:,.2f}")
print(f"  │")
print(f"  │  ! Sales values has a very wide range of ${range_sales:,.0f}")
print(f"  └──────────────────────────────────────────────────────────\n")

# --- VARIANCE ---
variance_sales = np.var(sales)            # population variance
variance_sales_sample = np.var(sales, ddof=1)  # sample variance

print("  ┌── VARIANCE ───────────────────────────────────────────────")
print(f"  │  Formula : Average of squared differences from mean")
print(f"  │  Population Variance = {variance_sales:,.2f}")
print(f"  │  Sample Variance     = {variance_sales_sample:,.2f}  (ddof=1)")
print(f"  │")
print(f"  │  ! High variance = sales values are very spread out")
print(f"  └──────────────────────────────────────────────────────────\n")
```

```
# --- STANDARD DEVIATION ---
std_sales = np.std(sales)
std_sample = np.std(sales, ddof=1)

print("  ┌── STANDARD DEVIATION ─────────────────────────────────────")
print(f"  │  Formula : Square root of Variance")
print(f"  │  Population Std Dev = ${std_sales:,.2f}")
print(f"  │  Sample Std Dev     = ${std_sample:,.2f}")
print(f"  │")
print(f"  │  68-95-99.7 Rule:")
print(f"  │    68% of orders fall between:")
print(f"  │      ${mean_sales - std_sales:,.2f}  and  ${mean_sales + std_sales:,.2f}")
print(f"  │    95% of orders fall between:")
print(f"  │      ${max(0, mean_sales - 2*std_sales):,.2f}  and  ${mean_sales + 2*std_sales:,.2f}")
print(f"  │")
print(f"  │  ! A typical order varies ±${std_sales:.2f} from the mean")
print(f"  └──────────────────────────────────────────────────────────\n")

# --- PERCENTILES & IQR ---
q1    = np.percentile(sales, 25)
q3    = np.percentile(sales, 75)
iqr   = q3 - q1

print("  ┌── PERCENTILES & IQR ──────────────────────────────────────")
print(f"  │  Q1 (25th percentile) = ${q1:.2f}")
print(f"  │  Q2 (50th percentile) = ${np.percentile(sales,50):.2f}  ← Median")
print(f"  │  Q3 (75th percentile) = ${q3:.2f}")
print(f"  │  IQR = Q3 - Q1        = ${iqr:.2f}")
print(f"  │")
print(f"  │  Outlier Fences:")
print(f"  │    Lower = Q1 - 1.5×IQR = ${q1 - 1.5*iqr:.2f}")
print(f"  │    Upper = Q3 + 1.5×IQR = ${q3 + 1.5*iqr:.2f}")
outliers = np.sum((sales < q1 - 1.5*iqr) | (sales > q3 + 1.5*iqr))
print(f"  │  Outliers detected: {outliers:,} orders")
print(f"  └──────────────────────────────────────────────────────────")
```

MODULE 3 · MEASURES OF DISPERSION

PART 1

CODE

```
MODULE 2 · Measures of Dispersion

📖 WHAT IS DISPERSION?
   It tells us HOW SPREAD OUT the data is.
   Even if two datasets have the same mean,
   their spread can be very different!

┌── RANGE ──────────────────────────────────────
│  Formula : Max – Min
│  Max Sales   = $3,400.00
│  Min Sales   = $2.99
│  Range       = $3,397.01
│
│  ! Sales values has a very wide range of $3,397
└───────────────────────────────────────────────

┌── VARIANCE ───────────────────────────────────
│  Formula : Average of squared differences from mean
│  Population Variance = 110,834.98
│  Sample Variance     = 110,835.57  (ddof=1)
│
│  ! High variance = sales values are very spread out
└───────────────────────────────────────────────
```

```
┌── STANDARD DEVIATION ─────────────────────────
│  Formula : Square root of Variance
│  Population Std Dev = $332.92
│  Sample Std Dev     = $332.92
│
│  68-95-99.7 Rule:
│    68% of orders fall between:
│      $-147.43  and  $518.41
│    95% of orders fall between:
│      $0.00  and  $851.33
│
│  ! A typical order varies ±$332.92 from the mean
└───────────────────────────────────────────────

┌── PERCENTILES & IQR ──────────────────────────
│  Q1 (25th percentile) = $11.95
│  Q2 (50th percentile) = $14.95  ← Median
│  Q3 (75th percentile) = $150.00
│  IQR = Q3 - Q1        = $138.05
│
│  Outlier Fences:
│    Lower = Q1 - 1.5×IQR = $-195.13
│    Upper = Q3 + 1.5×IQR = $357.08
│  Outliers detected: 37,027 orders
└───────────────────────────────────────────────
```

KEY FINDING

→Sales data shows a very wide spread between minimum and maximum order values.
→ High standard deviation indicates that customer purchase amounts vary significantly from the average sale.
→ Several outliers were detected, representing unusually high-value orders in the dataset.

MODULE 4 · MEASURES OF ASSOCIATION

PART 1

CODE

```
subheader("MODULE 3 · Measures of Association")

print("""
  @ WHAT IS ASSOCIATION?
     How two variables move TOGETHER.
     → COVARIANCE    : Direction of relationship (+/-)
     → CORRELATION   : Strength + Direction (-1 to +1)
""")

# Covariance
def cov(x, y):
    return np.sum((x - np.mean(x)) * (y - np.mean(y))) / (len(x) - 1)

cov_sales_price = cov(sales, price)
cov_sales_qty   = cov(sales, qty)
cov_price_qty   = cov(price, qty)

print("  COVARIANCE")
print(f"  {'—'*50}")
print(f"  Formula: Σ[(x - x̄)(y - ȳ)] ÷ (n-1)")
print(f"  Cov(Sales, Price)    = {cov_sales_price:>12.4f}")
print(f"  Cov(Sales, Quantity) = {cov_sales_qty:>12.4f}")
print(f"  Cov(Price, Quantity) = {cov_price_qty:>12.4f}")
print(f"""
  ! Covariance sign:
     Positive → both go UP together
     Negative → one goes UP, other goes DOWN
     Problem  → hard to compare (unit-dependent)
  → Use CORRELATION for better comparison!
""")
```

```
# Correlation
def corr(x, y):
    return cov(x, y) / (np.std(x, ddof=1) * np.std(y, ddof=1))

r_sp = corr(sales, price)
r_sq = corr(sales, qty)
r_pq = corr(price, qty)
r_sm = corr(sales, month)
r_sh = corr(sales, hour)

def corr_strength(r):
    a = abs(r)
    if a >= 0.8: return "Very Strong"
    if a >= 0.6: return "Strong"
    if a >= 0.4: return "Moderate"
    if a >= 0.2: return "Weak"
    return "Very Weak / No"

print("  PEARSON CORRELATION COEFFICIENT (r)")
print(f"  {'—'*60}")
print(f"  Formula: Cov(x,y) ÷ (Std_x × Std_y)   Range: -1 to +1")
print()
print(f"  {'Variables':<32} {'r':>8}  {'Direction':<10}  Strength")
print(f"  {'—'*60}")
for label, r in [
    ("Sales  vs  Price",    r_sp),
    ("Sales  vs  Quantity", r_sq),
    ("Price  vs  Quantity", r_pq),
    ("Sales  vs  Month",    r_sm),
    ("Sales  vs  Hour",     r_sh),
]:
    direction = "Positive ↑" if r > 0 else "Negative ↓"
    print(f"  {label:<32} {r:>+8.4f}  {direction:<10}  {corr_strength(r)}")

print(f"""
  ! Key Finding:
     Sales & Price → r = {r_sp:.4f}  (Very Strong Positive)
     → Higher priced products directly drive higher sales revenue
""")
```

MODULE 4 · MEASURES OF ASSOCIATION

PART 1

OUTPUT

```
MODULE 3 · Measures of Association

@ WHAT IS ASSOCIATION?
   How two variables move TOGETHER.
   → COVARIANCE   : Direction of relationship (+/-)
   → CORRELATION  : Strength + Direction (-1 to +1)

COVARIANCE
Formula: Σ[(x - x̄)(y - ȳ)] ÷ (n-1)
Cov(Sales, Price)    =  110684.5418
Cov(Sales, Quantity) =     -20.5521
Cov(Price, Quantity) =     -21.8451

! Covariance sign:
   Positive → both go UP together
   Negative → one goes UP, other goes DOWN
   Problem  → hard to compare (unit-dependent)
→ Use CORRELATION for better comparison!
```

```
PEARSON CORRELATION COEFFICIENT (r)

Formula: Cov(x,y) ÷ (Std_x x Std_y)   Range: -1 to +1

Variables                          r  Direction   Strength
Sales  vs  Price              +0.9992  Positive ↑  Very Strong
Sales  vs  Quantity           -0.1394  Negative ↓  Very Weak / No
Price  vs  Quantity           -0.1483  Negative ↓  Very Weak / No
Sales  vs  Month              -0.0035  Negative ↓  Very Weak / No
Sales  vs  Hour               +0.0017  Positive ↑  Very Weak / No

! Key Finding:
   Sales & Price → r = 0.9992  (Very Strong Positive)
   → Higher priced products directly drive higher sales revenue
```

KEY FINDING

→ Positive covariance between Sales and Price indicates that both variables increase together
→ A strong positive correlation was found between Sales and Price, showing that expensive products generate higher revenue.

MODULE 5 · MONTHLY SALES ANALYSIS

PART 2

CODE

```
month_names = ["Jan","Feb","Mar","Apr","May","Jun",
               "Jul","Aug","Sep","Oct","Nov","Dec"]

monthly_revenue = np.array([
    np.sum(sales[month == m]) for m in range(1, 13)
])
monthly_orders  = np.array([
    np.sum(month == m) for m in range(1, 13)
])

print(f"\n  {'Month':<6} {'Orders':>8} {'Revenue ($)':>14} {'Bar Chart'}")
print("  " + "—" * 50)

max_rev = np.max(monthly_revenue)
for i in range(12):
    bar = "▶" * int(monthly_revenue[i] / max_rev * 25)
    print(f"  {month_names[i]:<6} {monthly_orders[i]:>8,} "
          f"{monthly_revenue[i]:>14,.2f}  {bar}")

best_month = np.argmax(monthly_revenue)
worst_month = np.argmin(monthly_revenue)
print(f"\n  ⏳ Best Month  : {month_names[best_month]}  "
      f"(${monthly_revenue[best_month]:,.2f})")
print(f"  📉 Worst Month : {month_names[worst_month]}  "
      f"(${monthly_revenue[worst_month]:,.2f})")

# Month-over-Month growth
mom_growth = np.diff(monthly_revenue) / monthly_revenue[:-1] * 100
print(f"\n  Month-over-Month Growth (%):")
for i in range(11):
    arrow = "▲" if mom_growth[i] > 0 else "▼"
    print(f"    {month_names[i]} → {month_names[i+1]}: "
          f"{arrow} {mom_growth[i]:+.1f}%")
```

OUTPUT

```
  Month    Orders    Revenue ($) Bar Chart
  ——————————————————————————————————————————————————
  Jan       9,709    1,822,256.73 ▶▶▶▶▶▶▶▶▶
  Feb      11,975    2,202,022.42 ▶▶▶▶▶▶▶▶▶▶▶
  Mar      15,153    2,807,100.38 ▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶
  Apr      18,279    3,390,670.24 ▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶
  May      16,566    3,152,606.75 ▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶
  Jun      13,554    2,577,802.26 ▶▶▶▶▶▶▶▶▶▶▶▶▶
  Jul      14,293    2,647,775.76 ▶▶▶▶▶▶▶▶▶▶▶▶▶▶
  Aug      11,961    2,244,467.88 ▶▶▶▶▶▶▶▶▶▶▶▶
  Sep      11,621    2,097,560.13 ▶▶▶▶▶▶▶▶▶▶▶▶
  Oct      20,282    3,736,726.88 ▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶
  Nov      17,573    3,199,603.20 ▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶
  Dec      24,984    4,613,443.34 ▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶

  ⏳ Best Month  : Dec  ($4,613,443.34)
  📉 Worst Month : Jan  ($1,822,256.73)

  Month-over-Month Growth (%):
    Jan → Feb: ▲ +20.8%
    Feb → Mar: ▲ +27.5%
    Mar → Apr: ▲ +20.8%
    Apr → May: ▼ -7.0%
    May → Jun: ▼ -18.2%
    Jun → Jul: ▲ +2.7%
    Jul → Aug: ▼ -15.2%
    Aug → Sep: ▼ -6.5%
    Sep → Oct: ▲ +78.1%
    Oct → Nov: ▼ -14.9%
    Nov → Dec: ▲ +44.2%
```

MODULE 6 · PRODUCT PERFORMANCE

PART 2

CODE

```
subheader("  MODULE 2 · PRODUCT PERFORMANCE")
products = np.unique(product)
prod_rev  = np.array([np.sum(sales[product == p])    for p in products])
prod_qty  = np.array([np.sum(qty[product == p])      for p in products])
prod_cnt  = np.array([np.sum(product == p)           for p in products])

# Sort by revenue (highest first)
sort_idx = np.argsort(prod_rev)[::-1]

print(f"\n  {'#':<3} {'Product':<30} {'Revenue ($)':>13} "
      f"{'Units':>8} {'Orders':>8}")
print("  " + "—" * 66)
for rank, i in enumerate(sort_idx, 1):
    print(f"  {rank:<3} {products[i]:<30} "
          f"{prod_rev[i]:>13,.2f} "
          f"{prod_qty[i]:>8,} "
          f"{prod_cnt[i]:>8,}")

print(f"\n  🥇 Top Revenue Product : {products[sort_idx[0]]}")
print(f"  📦 Most Units Sold     : "
      f"{products[np.argmax(prod_qty)]}")
print(f"  🔢 Most Orders         : "
      f"{products[np.argmax(prod_cnt)]}")

# Revenue share of top 3
top3_share = np.sum(prod_rev[sort_idx[:3]]) / np.sum(prod_rev) * 100
print(f"  📊 Top 3 Products = {top3_share:.1f}% of total revenue")
```

OUTPUT

```
  MODULE 2 · PRODUCT PERFORMANCE

  #   Product                          Revenue ($)    Units   Orders
  ——————————————————————————————————————————————————————————————
  1   Macbook Pro Laptop             8,037,600.00    4,728    4,724
  2   iPhone                         4,794,300.00    6,849    6,842
  3   ThinkPad Laptop                4,129,958.70    4,130    4,128
  4   Google Phone                   3,319,200.00    5,532    5,525
  5   27in 4K Gaming Monitor         2,435,097.56    6,244    6,230
  6   34in Ultrawide Monitor         2,355,558.01    6,199    6,181
  7   Apple Airpods Headphones       2,349,150.00   15,661   15,549
  8   Flatscreen TV                  1,445,700.00    4,819    4,800
  9   Bose SoundSport Headphones     1,345,565.43   13,457   13,325
  10  27in FHD Monitor               1,132,424.50    7,550    7,507
  11  Vareebadd Phone                  827,200.00    2,068    2,065
  12  20in Monitor                     454,148.71    4,129    4,101
  13  LG Washing Machine               399,600.00      666      666
  14  LG Dryer                         387,600.00      646      646
  15  Lightning Charging Cable         347,094.15   23,217   21,658
  16  USB-C Charging Cable             286,501.25   23,975   21,903
  17  Wired Headphones                 246,478.43   20,557   18,882
  18  AA Batteries (4-pack)            106,118.40   27,635   20,577
  19  AAA Batteries (4-pack)            92,740.83   31,017   20,641

  🥇 Top Revenue Product : Macbook Pro Laptop
  📦 Most Units Sold     : AAA Batteries (4-pack)
  🔢 Most Orders         : USB-C Charging Cable
  📊 Top 3 Products = 49.2% of total revenue
```

KEY FINDING

→ Product performance analysis identified the top revenue-generating and most frequently sold products.
→ evenue, quantity sold, and total orders were calculated for each product category.
→ The top 3 products contributed a significant percentage of total company revenue.

MODULE 7 · CITY-WISE SALES

PART 2

CODE

```
subheader("  MODULE 3 · CITY-WISE SALES")

cities    = np.unique(city)
city_rev  = np.array([np.sum(sales[city == c]) for c in cities])
city_cnt  = np.array([np.sum(city == c)        for c in cities])

city_sort = np.argsort(city_rev)[::-1]
max_city_rev = np.max(city_rev)

print(f"\n  {'City':<16} {'Orders':>8} {'Revenue ($)':>14} {'Share':>7}  Bar")
print("  " + "—" * 60)
total_rev = np.sum(sales)
for i in city_sort:
    share = city_rev[i] / total_rev * 100
    bar   = "▶" * int(city_rev[i] / max_city_rev * 20)
    print(f"  {cities[i]:<16} {city_cnt[i]:>8,} "
          f"{city_rev[i]:>14,.2f} {share:>6.1f}%  {bar}")

print(f"\n  ! Top City : {cities[city_sort[0]]}  "
      f"(${city_rev[city_sort[0]]:,.2f})")
print(f"  ! Least City: {cities[city_sort[-1]]}  "
      f"(${city_rev[city_sort[-1]]:,.2f})")
```

OUTPUT

```
──────────────────────────────────────────────────────────────────────

  MODULE 3 · CITY-WISE SALES

──────────────────────────────────────────────────────────────────────

  City             Orders    Revenue ($)   Share  Bar
  ────────────────────────────────────────────────────────────
  San Francisco    44,732   8,262,203.91   24.0%  ▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶▶
  Los Angeles      29,605   5,452,570.80   15.8%  ▶▶▶▶▶▶▶▶▶▶▶▶
  New York City    24,876   4,664,317.43   13.5%  ▶▶▶▶▶▶▶▶▶▶▶
  Boston           19,934   3,661,642.01   10.6%  ▶▶▶▶▶▶▶▶
  Atlanta          14,881   2,795,498.58    8.1%  ▶▶▶▶▶▶
  Dallas           14,820   2,767,975.40    8.0%  ▶▶▶▶▶▶
  Seattle          14,732   2,747,755.48    8.0%  ▶▶▶▶▶▶
  Portland         12,465   2,320,490.61    6.7%  ▶▶▶▶▶
  Austin            9,905   1,819,581.75    5.3%  ▶▶▶▶

  ! Top City : San Francisco  ($8,262,203.91)
  ! Least City: Austin  ($1,819,581.75)
```

KEY FINDING

→ City-wise sales analysis identified the highest and lowest revenue-generating cities.
→ Revenue contribution and total order counts were calculated for each city.
→ The top-performing city contributed the largest share of overall company sales..

MODULE 8 · FINAL SUMMERY

PART 2

CODE

```
subheader("  MODULE 4 · FINAL SUMMARY DASHBOARD")

print(f"""
   💰 Total Revenue          : ${total_rev:>14,.2f}
   📦 Total Units Sold       : {np.sum(qty):>14,}
   📄 Total Orders           : {N:>14,}
   🛍️ Unique Products        : {len(products):>14}
   🌆 Cities Covered         : {len(cities):>14}

   📊 Avg Sale per Order     : ${np.mean(sales):>14,.2f}
   📊 Median Sale            : ${np.median(sales):>14,.2f}
   📊 Std Dev of Sales       : ${np.std(sales):>14,.2f}
   📊 Avg Units per Order    : {np.mean(qty):>14.2f}

   ⏳ Best Month             : {month_names[best_month]} 2019
   🌆 Top City               : {cities[city_sort[0]]}
   🥇 Top Revenue Product    : {products[sort_idx[0]]}
   📦 Most Units Sold        : {products[np.argmax(prod_qty)]}
""")

# Percentile table
print("  Sales Percentiles:")
for p_val in [25, 50, 75, 90, 95, 99]:
    print(f"    P{p_val:<3} → ${np.percentile(sales, p_val):>10,.2f}")
```

OUTPUT

```
──────────────────────────────────────────────────────────────────────
   MODULE 4 · FINAL SUMMARY DASHBOARD
──────────────────────────────────────────────────────────────────────

   💰 Total Revenue          : $ 34,492,035.97
   📦 Total Units Sold       :         209,079
   📄 Total Orders           :         185,950
   🛍️ Unique Products        :              19
   🌆 Cities Covered         :               9

   📊 Avg Sale per Order     : $         185.49
   📊 Median Sale            : $          14.95
   📊 Std Dev of Sales       : $         332.92
   📊 Avg Units per Order    :            1.12

   ⏳ Best Month             : Dec 2019
   🌆 Top City               : San Francisco
   🥇 Top Revenue Product    : Macbook Pro Laptop
   📦 Most Units Sold        : AAA Batteries (4-pack)

  Sales Percentiles:
    P25  → $      11.95
    P50  → $      14.95
    P75  → $     150.00
    P90  → $     600.00
    P95  → $     700.00
    P99  → $   1,700.00
```

KEY FINDING

→ The summary dashboard provides a complete overview of total revenue, orders, units sold, products, and cities covered.
→ Key business insights such as best month, top city, and highest revenue-generating product were identified.
→ Statistical measures like average sales, median sales, and standard deviation helped understand overall sales behavior.
→ Percentile analysis revealed the distribution of customer order values and highlighted high-value sales trends.

KEY INSIGHTS & CONCLUSIONS

Data Distribution

RIGHT-SKEWED — Mean $185 >> Median $14.95. Most orders are cheap cables/batteries, few are expensive laptops.

Top Performer

Macbook Pro Laptop leads with $8.03M revenue (23.3% of total). Top 3 products = 48.9% of all revenue.

City Dominance

San Francisco generates 24% of total revenue ($8.26M) — 4.5× more than least-performing Austin.

Seasonality

December is the peak month at $4.61M. Q4 (Oct–Dec) drives the highest sales — holiday season effect.

Correlation

Sales & Price show r = +0.9992 (near-perfect). Price is the single strongest predictor of revenue.

NumPy Power

All analysis done using NumPy only — np.mean, np.std, np.corrcoef, np.percentile, np.argsort, np.unique.

THANK YOU

Presented by : Dhruv dabhi